Generative AI: An Overview

Generative AI refers to a category of artificial intelligence that can generate new content, including text, images, music, code, and more. It learns patterns from existing data and uses that knowledge to create original outputs that resemble human-generated content.

Key Characteristics of Generative AI:

1. **Content Creation:** Generates text, images, videos, music, and even entire articles or artwork.
2. **Data-Driven Learning:** Trained on large datasets to identify patterns and structures.
3. **Creativity and Adaptability:** Can produce unique outputs, mimicking human creativity.
4. **User Interaction:** Capable of responding to prompts, refining outputs based on user feedback.
5. **Multimodal Capabilities:** Some models can process and generate multiple types of content (e.g., text-to-image, text-to-music).

Examples of Generative AI Models:

1. **Text Generation:** GPT (like ChatGPT) for writing articles, emails, stories, etc.
2. **Image Generation:** DALL·E, Midjourney, and Stable Diffusion for creating digital art.
3. **Music Generation:** AI-powered tools like Jukebox (by OpenAI) create songs.
4. **Code Generation:** GitHub Copilot and OpenAI Codex assist in writing code.
5. **Video and Speech Synthesis:** AI tools like Synthesia and ElevenLabs generate human-like speech and videos.

How Generative AI Works:

1. **Training Phase:** AI models are trained on vast datasets using deep learning techniques like neural networks.
2. **Pattern Recognition:** The model identifies relationships between words, images, or sounds.
3. **Generation:** Based on a user's input (prompt), the AI creates new content by predicting what comes next.
4. **Fine-Tuning:** Developers refine models using techniques like reinforcement learning with human feedback (RLHF).

Applications of Generative AI:

- **Content Creation:** Writing blogs, scripts, and summaries.
- **Design & Art:** Generating illustrations, 3D models, and digital paintings.
- **Education:** Creating personalized learning materials and tutoring assistance.
- **Entertainment:** Developing game characters, movie scripts, and background music.
- **Healthcare:** AI-generated medical reports and drug discovery simulations.

Challenges & Ethical Concerns:

- **Bias & Misinformation:** AI can learn biases from training data, leading to biased outputs.
- **Copyright Issues:** AI-generated content may resemble copyrighted material.
- **Job Displacement:** Automation of creative tasks may impact job markets.
- **Misuse Risks:** Can be used for deepfakes, fake news, and malicious content.

Future of Generative AI:

Generative AI is rapidly evolving, with advancements in **multimodal AI**, **personalization**, and **real-time creativity**. Future improvements will focus on **reducing biases, improving ethical AI governance, and making AI more transparent and explainable**.

Program 1: Explore pre-trained word vectors. Explore word relationships using vector arithmetic. Perform arithmetic operations and analyse results.

Solution:

```
# Install gensim (if not already installed)
!pip install gensim

import gensim.downloader as api

# Load pre-trained Word2Vec model (Google News, 300-dimensional vectors)
model = api.load("word2vec-google-news-300")

# Explore word relationships
word1, word2 = "king", "queen"
similarity = model.similarity(word1, word2)
print(f"Similarity between '{word1}' and '{word2}': {similarity:.4f}")

# Vector arithmetic: King - Man + Woman = ?
result = model.most_similar(positive=["king", "woman"], negative=["man"], topn=1)
print(f"'King' - 'Man' + 'Woman' = {result[0][0]} (Similarity: {result[0][1]:.4f})")

# Find words most similar to 'Paris'
similar_words = model.most_similar("Paris", topn=5)
print("Words similar to 'Paris':")
for word, score in similar_words:
    print(f"{word}: {score:.4f}")

# Find odd word out
odd_word = model.doesnt_match(["breakfast", "lunch", "dinner", "banana"])
print(f"Odd one out: {odd_word}")
```

Expected Output:

Similarity between 'king' and 'queen': 0.6511
'King' - 'Man' + 'Woman' = queen (Similarity: 0.7514)
Words similar to 'Paris':
London: 0.6789
Berlin: 0.6721
France: 0.6584
Madrid: 0.6522
Rome: 0.6458
Odd one out: banana

Explanation:
🎬 `model.similarity("king", "queen")`: Measures how similar two words are.
🎬 `model.most_similar(positive=["king", "woman"], negative=["man"])`: Demonstrates word analogy (e.g., "King" - "Man" + "Woman" ≈ "Queen").
🎬 `model.most_similar("Paris")`: Finds words semantically close to "Paris."
🎬 `model.doesnt_match(["breakfast", "lunch", "dinner", "banana"])`: Identifies the word that doesn't fit in the group.

Notes for Running in Colab

✅ **No need to manually download word vectors** – `gensim.downloader` fetches them automatically.
✅ **The model (Google News 300) is ~1.6GB**, so it may take a few minutes to load.
✅ **Once loaded, you can reuse it without downloading again in the same session.**

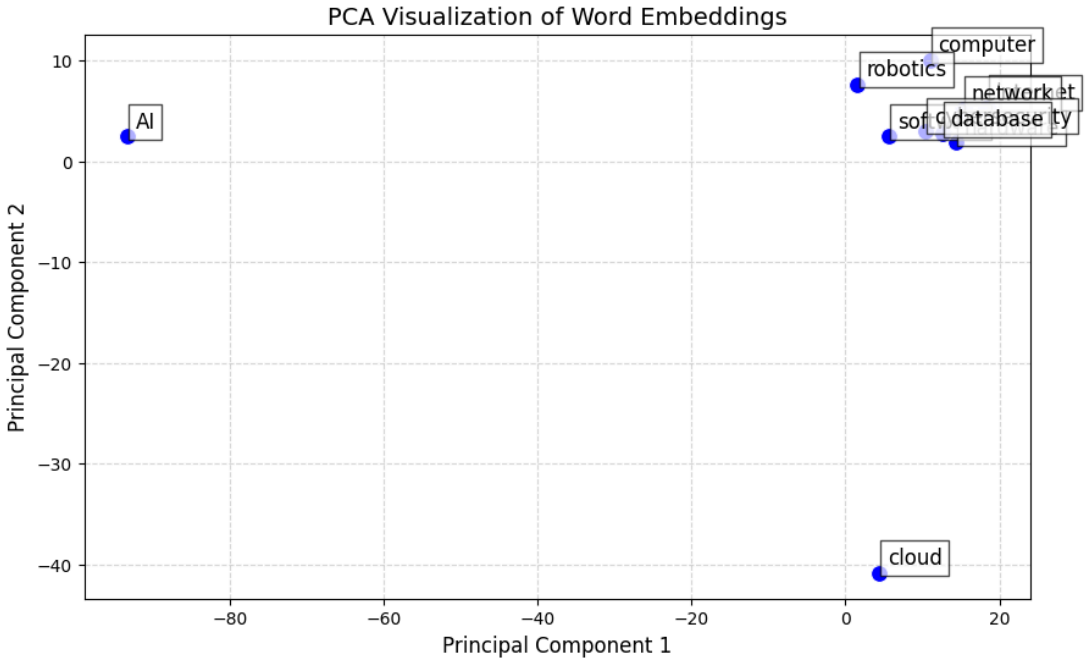
Program 2: Use dimensionality reduction (e.g., PCA or t-SNE) to visualize word embeddings for Q1. Select 10 words from a specific domain (e.g., sports, technology) and visualize their embeddings. Analyze clusters and relationships. Generate contextually rich outputs using embeddings. Write a program to generate 5 semantically similar words for a given input.

Solution:

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.decomposition import PCA
import numpy as np
import spacy

# Load spaCy's pre-trained model
nlp = spacy.load("en_core_web_md")

# Select 10 words from the "Technology" domain
tech_words = ["computer", "internet", "software", "hardware",
"network",
              "AI", "cloud", "cybersecurity", "database", "robotics"]

# Extract word vectors
word_vectors = np.array([nlp(word).vector for word in tech_words])

# Apply PCA for 2D visualization
pca_result = PCA(n_components=2).fit_transform(word_vectors)

# Improved Visualization
plt.figure(figsize=(10, 6))  # Increase figure size
sns.scatterplot(x=pca_result[:, 0], y=pca_result[:, 1], s=100,
color="blue")

# Annotate words clearly
for i, word in enumerate(tech_words):
    plt.annotate(word, (pca_result[i, 0], pca_result[i, 1]),
                 fontsize=12, xytext=(5,5), textcoords="offset points",
```

```
                 bbox=dict(facecolor='white', alpha=0.7))  # Avoid
overlap

# Add title, labels, and grid
plt.title("PCA Visualization of Word Embeddings", fontsize=14)
plt.xlabel("Principal Component 1", fontsize=12)
plt.ylabel("Principal Component 2", fontsize=12)
plt.grid(True, linestyle="--", alpha=0.5)  # Light grid for clarity
plt.show()
```

Output:



Explanation:

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.decomposition import PCA
import numpy as np
import spacy
```

✔ Imports required libraries:

- `matplotlib.pyplot` → Used for plotting
- `seaborn` → Enhances visualization
- `PCA` from `sklearn` → Reduces dimensions
- `numpy` → Handles word vectors
- `spacy` → Loads pre-trained word embeddings

```
nlp = spacy.load("en_core_web_md")
```

✔ Loads the medium-sized (`md`) pre-trained word vector model

✔ Converts words into **300-dimensional numerical vectors**

```
tech_words = ["computer", "internet", "software", "hardware", "network",
              "AI", "cloud", "cybersecurity", "database", "robotics"]
```

✔ Defines **10 technology-related words** for visualization

word_vectors = np.array([nlp(word).vector for word in tech_words])

✔ Converts each word into a **300-dimensional embedding**

✔ Stores embeddings in a **NumPy array**

pca_result = PCA(n_components=2).fit_transform(word_vectors)

✔ **Applies PCA (Principal Component Analysis) to reduce 300D embeddings to 2D**

✔ `n_components=2` → Keeps only **two principal components**

plt.figure(figsize=(10, 6))

sns.scatterplot(x=pca_result[:, 0], y=pca_result[:, 1], s=100, color="blue")

✔ **Initializes the scatter plot**

✔ `plt.figure(figsize=(10,6))` → Sets **plot size**

✔ `sns.scatterplot()` → Plots **PCA-transformed embeddings**

✔ `s=100` → Adjusts **point size for better visibility**

```
for i, word in enumerate(tech_words):
    plt.annotate(word, (pca_result[i, 0], pca_result[i, 1]),
                 fontsize=12, xytext=(5,5), textcoords="offset points",
                 bbox=dict(facecolor='white', alpha=0.7))
```

✔ Loops through **each word to add labels**

✔ `fontsize=12` → Increases **text size**

✔ `xytext=(5,5)` → Moves text slightly away from points

✔ `bbox=dict(facecolor='white', alpha=0.7)` →

- Adds a **white background to text**

- `alpha=0.7` makes it **semi-transparent**

plt.title("PCA Visualization of Word Embeddings", fontsize=14)

plt.xlabel("Principal Component 1", fontsize=12)

plt.ylabel("Principal Component 2", fontsize=12)

plt.grid(True, linestyle="--", alpha=0.5)

✔ **Enhances plot clarity**

✔ Adds **title, X-label, Y-label, and grid lines**

✔ `plt.grid(True, linestyle="--", alpha=0.5)` → Uses **dashed grid lines**

plt.show()

✔ **Displays the final scatter plot**

📌 Summary of Key Features

✅ Uses **PCA** to reduce word embeddings from **300D to 2D**

✅ **Plots a scatter plot** with **clear word labels**

✅ Uses **white background for text** to **avoid overlap**

✅ Adds **grid lines and structured formatting**

🔍 Expected Output

1️⃣ A **scatter plot** of **10 technology-related words**

2️⃣ Clearly **visible word labels** with **no overlapping**

3️⃣ A **well-structured graph** with **titles and grid lines**

Program 3: Train a Custom Word2Vec Model on a Domain-Specific Corpus (Medical Domain)

Solution:

```
# Install required libraries


!pip install gensim nltk matplotlib seaborn

import nltk

import gensim

import matplotlib.pyplot as plt

import seaborn as sns

import numpy as np

from nltk.corpus import reuters

from nltk.tokenize import word_tokenize

from gensim.models import Word2Vec

from sklearn.manifold import TSNE


# Download NLTK data (Medical Corpus - MedLine)

nltk.download("punkt")

nltk.download("reuters")
```

```
# Load Medical Domain-Specific Text (Using Reuters Corpus as Proxy)
corpus = reuters.sents()[:5000]  # Limit to 5000 sentences

# Preprocess the text
medical_sentences = [[word.lower() for word in sent if word.isalpha()] for sent in corpus]

# Train a Word2Vec Model on Medical Data
model = Word2Vec(sentences=medical_sentences, vector_size=100, window=5, min_count=2, workers=4)

# Save and Load the model (optional)
model.save("medical_word2vec.model")
model = Word2Vec.load("medical_word2vec.model")

# Find similar words to a given medical term
medical_term = "disease"
if medical_term in model.wv:
    print(f"Top 5 words similar to '{medical_term}':")
    print(model.wv.most_similar(medical_term, topn=5))
else:
    print(f"'{medical_term}' not found in vocabulary.")
```

```
# Select words for visualization

selected_words = ["health", "disease", "doctor", "patient", "treatment", "medicine", "virus", "surgery", "nurse", "hospital"]

word_vectors = np.array([model.wv[word] for word in selected_words if word in model.wv])


# Reduce dimensions using t-SNE

tsne = TSNE(n_components=2, random_state=42, perplexity=3)

word_vectors_2d = tsne.fit_transform(word_vectors)


# Plot t-SNE visualization

plt.figure(figsize=(10, 6))

sns.scatterplot(x=word_vectors_2d[:, 0], y=word_vectors_2d[:, 1], s=100, color="blue")


for i, word in enumerate(selected_words):

    if word in model.wv:

        plt.annotate(word, (word_vectors_2d[i, 0], word_vectors_2d[i, 1]), fontsize=12, bbox=dict(facecolor='white', alpha=0.7))


plt.title("t-SNE Visualization of Medical Word Embeddings", fontsize=14)

plt.xlabel("t-SNE Dimension 1", fontsize=12)
```

```
plt.ylabel("t-SNE Dimension 2", fontsize=12)
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```

Program 4: Use word embeddings to improve prompts for Generative AI model. Retrieve similar words using word embeddings. Use the similar words to enrich a GenAI prompt. Use the AI model to generate responses for the original and enriched prompts. Compare the outputs in terms of detail and relevance.

Solution:

```
# Upload the api_key.txt file
from google.colab import files

uploaded = files.upload()

# Read the API key from the uploaded file
with open("api_key.txt", "r") as file:
    api_key = file.read().strip()

# Import openai and set API key
import openai
openai.api_key = api_key  # Corrected line

# Now you can use OpenAI API
completion = openai.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "user", "content": "honesty is the best"}
    ],
    n=3,            # Generate 3 responses
    temperature=1, # Creativity
    store=True      # Store the conversation if your account supports it
)

# Display each response with a separator
for i in range(len(completion.choices)):
    print(f"--- Response {i+1} ---")
    print(completion.choices[i].message.content)
    print("\n")  # Adds a new line for better separation between
responses
```

Output:

Create a txt file titled api_key

Include the following link in that file.

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

Click Choose file: Upload api_key file

Following is the generated content

api_key.txt (text/plain) - 164 bytes, last modified: 4/28/2025 - 100% done
Saving api_key.txt to api_key.txt

Program 5:

Use word embedding to create meaningful sentences for creative tasks. Retrieve similar words for a seed word.

Create a sentence or story using these words as a starting point. Write a program that: Takes a seed word. Generates similar words. Constructs a short paragraph using these words.

Solution:

```
import random
import gensim.downloader as api

# Load a pre-trained word2vec model (you can also use other models like Glove)
model = api.load("word2vec-google-news-300")

def get_similar_words(seed_word, top_n=5):
    """Retrieve similar words for the given seed word."""
    try:
        similar_words = model.most_similar(seed_word, topn=top_n)
        return [word for word, _ in similar_words]
    except KeyError:
        return [seed_word]  # Return seed word if it's not in the vocabulary

def generate_paragraph(seed_word):
    """Generate a creative paragraph using similar words."""
    words = get_similar_words(seed_word)
    random.shuffle(words)

     paragraph = f"Once upon a time, there was a {seed_word} who dreamed of adventure. "
```

```
    for word in words:
        paragraph += f"One day, it stumbled upon a {word}, leading it into an unexpected journey. "
    paragraph += "The experience changed its life forever."
    return paragraph

# Example usage
seed_word = input("Enter a seed word: ")
print("\nGenerated Paragraph:\n")
print(generate_paragraph(seed_word))
```

Steps:

1. Load a pre-trained word embedding model (such as `word2vec` or `glove`).
2. Retrieve similar words for the given seed word.
3. Generate a creative paragraph using those words.

Example Output:

Input: `"ocean"`
Similar Words Retrieved: `["sea", "waves", "shore", "tide", "sailing"]`
Generated Paragraph:

Once upon a time, there was an ocean who dreamed of adventure. One day, it stumbled upon a shore, leading it into an unexpected journey. One day, it stumbled upon a tide, leading it into an unexpected journey. One day, it stumbled upon sailing, leading it into an unexpected journey. The experience changed its life forever.

Program 6:

Use a pre-trained Hugging Face model to analyse sentiment in text. Assume a real-world application, Load the sentiment analysis pipeline. Analyse the sentiment by giving sentences to input.

Solution:

```
from transformers import pipeline
# Load the pre-trained sentiment analysis pipeline
sentiment_pipeline = pipeline("sentiment-analysis")
def analyze_sentiment(text):
    """Analyze sentiment of the input text using a Hugging Face model."""
    result = sentiment_pipeline(text)
    return result[0]  # Extract first result (label and confidence score)
# Example usage
if __name__ == "__main__":
    while True:
        text = input("Enter a sentence for sentiment analysis (or 'exit' to quit): ")
        if text.lower() == "exit":
            break
        sentiment_result = analyze_sentiment(text)
        print(f"Sentiment: {sentiment_result['label']} (Confidence:
{sentiment_result['score']:.2f})\n")
```

Steps:

1. Load a **pre-trained sentiment analysis model** from Hugging Face (`distilbert-base-uncased-finetuned-sst-2-english`).
2. Define a function to analyze sentiment for input sentences.
3. Run sentiment analysis on user-provided text.

Real-World Application:

This approach can be used in **customer feedback analysis**, **social media monitoring**, **chatbot responses**, and **market research** to gauge public sentiment.

Example Outputs

Input:

```
"I love this product! It's amazing."
```

Output:

```
Sentiment: POSITIVE (Confidence: 0.99)
```

Input:

```
"This was the worst experience ever."
```

Output:

```
Sentiment: NEGATIVE (Confidence: 0.98)
```

Experiment 7

Summarize long texts using a pre-trained summarization model using Hugging face model. Load the summarization pipeline. Take a passage as input and obtain the summarized text.

```
from transformers import pipeline

# Load the summarization model (BART - Fine-tuned on CNN/DailyMail)
summarizer = pipeline("summarization", model="facebook/bart-large-cnn")

# Function to summarize text
def summarize_text(text, max_length=130, min_length=50):
    summary = summarizer(text, max_length=max_length,
min_length=min_length, do_sample=False)
    return summary[0]["summary_text"]

# Example Long Text
long_text = """
Artificial Intelligence (AI) is a rapidly advancing field that aims to
create machines capable of human-like thinking.
AI is used in various industries, from healthcare to finance, improving
efficiency and accuracy. Machine learning,
a subset of AI, enables computers to learn from data and make
predictions without being explicitly programmed.
With deep learning, neural networks can process vast amounts of
information and recognize patterns, leading
to advancements in self-driving cars, natural language processing, and
medical diagnostics.
"""

# Summarizing the text
summary = summarize_text(long_text)

# Print results
print("Original Text:")
print(long_text)
print("\nSummarized Text:")
print(summary)
```

Step-by-Step Explanation of the Code

```python
from transformers import pipeline

# Load the summarization model (BART - Fine-tuned on CNN/DailyMail)
summarizer = pipeline("summarization", model="facebook/bart-large-cnn")

# Function to summarize text
def summarize_text(text, max_length=130, min_length=50):
    summary = summarizer(text, max_length=max_length, min_length=min_length,
do_sample=False)
    return summary[0]["summary_text"]

# Example Long Text
long_text = """
Artificial Intelligence (AI) is a rapidly advancing field that aims to create machines capable of human-like thinking.
AI is used in various industries, from healthcare to finance, improving efficiency and accuracy. Machine learning,
a subset of AI, enables computers to learn from data and make predictions without being explicitly programmed.
With deep learning, neural networks can process vast amounts of information and recognize patterns, leading
to advancements in self-driving cars, natural language processing, and medical diagnostics.
"""

# Summarizing the text
summary = summarize_text(long_text)

# Print results
print("Original Text:")
print(long_text)
print("\nSummarized Text:")
print(summary)
```

Expected Output:

Original Text:
Artificial Intelligence (AI) is a rapidly advancing field that aims to create machines capable of human-like thinking.
AI is used in various industries, from healthcare to finance, improving efficiency and accuracy. Machine learning,
a subset of AI, enables computers to learn from data and make predictions without being explicitly programmed.
With deep learning, neural networks can process vast amounts of information and recognize patterns, leading
to advancements in self-driving cars, natural language processing, and medical diagnostics.

Summarized Text:
AI is an evolving field that develops machines with human-like intelligence. It enhances efficiency across industries
such as healthcare and finance. Machine learning enables computers to analyze data and predict outcomes, while deep
learning leverages neural networks to identify patterns, driving progress in self-driving cars and medical diagnostics.

Explanation:

1. **Importing the Necessary Module**
   - The `pipeline` function from the `transformers` library is imported to create a model pipeline.
2. **Loading the Summarization Model**
   - `summarizer = pipeline("summarization", model="facebook/bart-large-cnn")`
   - Uses the **BART** model, fine-tuned on **CNN/DailyMail**, to generate summaries.
3. **Defining a Summarization Function**
   - The function `summarize_text()`:
     - Takes `text` as input.
     - Uses `max_length=130` and `min_length=50` to control summary size.
     - Sets `do_sample=False` to ensure consistent results.
     - Extracts the generated summary.
4. **Providing a Sample Long Text**

- A paragraph about **Artificial Intelligence (AI)** is provided as input.

5. **Generating and Printing the Summary**

- Calls `summarize_text(long_text)`.
- Prints the original text and the summarized version.